

SOP 30 — Build Daily Payload

Camada A.N.I.: Instruments-Tools (build_daily_payload.py) · Versão: 1.0 (Protocol 0, 2026-09-09)
Golden Rule: mudou a lógica → atualiza o SOP → só então o código.

Objetivo

Montar o **Daily Payload** (`DeliveryPayload.Daily`) do dia a partir dos cards finalizados (SOP 21), do estado do jogo (SOP 11), da curadoria social (SOP 12) e dos venues (SOP 60) — com as **4 seções obrigatórias** (`agora` , `nacao` , `cortes` , `perto_de_voce`) e todas as regras de conteúdo do blueprint. A montagem é 100% determinística; nenhuma decisão editorial nova aqui.

Quando roda

- Cron daily build 07:15 (payload principal, `generated_at` 07:15 –03:00).
- Refreshes 12:00/18:00: rebuild apenas se houver cards novos aprovados na fila ou mudança de jogo.

Input

- Cards do dia: `.tmp/cards_final/<YYYY-MM-DD>.json` (SOP 21).
- Match: `.tmp/matches/<match_id>.json` (SOP 11) — pode ser `null`.
- Social (MVP): itens curados de `.tmp/social/<YYYY-MM-DD>.json` (SOP 12) — pode não existir.
- Venues: lista da aba `venues` filtrada (SOP 60).
- Flags: `AUTO_PUBLISH` (`.env`), cidade default (`Rio de Janeiro` no MVP).

Output

- `DeliveryPayload.Daily` → `.tmp/payload/<YYYY-MM-DD>.json` com `status: "draft"`.
- Valida contra o schema da CONSTITUTION **antes** de sair da tool (validação local, sem LLM).

Passos determinísticos

1. Ler entradas; `date` = hoje em America/Sao_Paulo; `generated_at` = momento da montagem (–03:00).
2. Determinar `mode`: `matchday` se existir jogo `profissional_m` nas próximas 18 horas (kickoff dentro da janela); senão `normal`.
3. Montar `agora`:
 - `match`: objeto do jogo quando houver (matchday) — **match center no topo do agora** em dia de jogo; senão `null`.
 - `headline`: manchete do dia escolhida deterministicamente (card oficial mais recente → senão card mais recente sem `needs_review`; sem cards → `""`).
 - `cards`: cards válidos, ordenados (oficiais/confirmados primeiro, depois por `published_at` desc), **sem** `needs_review` e sem `duplicate_of`.
4. Montar `nacao`: até 5 itens curados; vazio → `status: "empty"`, `items: []`.

5. Montar `cortes` : `lances` (UGC com crédito ou material próprio — nunca FlaTV/Globo/ESPN) e `memes` (allowlist + UGC enviado); vazio → listas vazias.
6. Montar `perto_de_voce` : `default_city` = cidade default; `items` = venues `status=verified` da cidade default (até 5); vazio → `items: []`.
7. `editor_notes` : anotações automáticas (ex.: itens `needs_review` existentes hoje, fonte que falhou na ingest).
8. **Regra de ouro da estrutura:** seção vazia **nunca é omitida** — `items: []` + `status: "empty"` quando o campo tiver status.
9. Validar contra o schema; gravar `.tmp/payload/<YYYY-MM-DD>.json` ; reportar `{ "mode": "...", "cards": n, "sections_empty": [...] } .`

Edge cases

- Sem cards e sem jogo → payload ainda é montado (seções vazias); editor vê “nada hoje” e decide segurar.
- Card com `needs_review` → fora do payload; entra em `editor_notes` para o humano decidir (✎ pode promover).
- Dois jogos no mesmo dia → `agora.match` = jogo do `profissional_m` mais próximo; demais vão para `editor_notes`.
- Data de `date` divergente do `generated_at` (rodada pós-meia-noite) → usar a data da coleta (06:30) como `date` ; logar.
- Validação de schema falhou → **não gravar**; reportar erro e acionar SOP 90.

Rate limits / limites conhecidos

- Nenhum externo (processamento local). Limites de escrita na cloud aplicam-se no SOP 21/50.

Testes

- Fixture: montar payload a partir de `fixtures/news_bundle.json` processado + `fixtures/match_scheduled.json` → `mode: "matchday"` , `agora.match` preenchido, 4 seções presentes; e com `match_live_goal.json` → placar refletido.
- Validar sempre contra o schema `DeliveryPayload.Daily` (JSON Schema local).